

HomiCSx: An end-to-end FEniCSx framework for finite-element computational homogenization

Amir Reza Tahouni*

Independent researcher, Tehran, Iran

Abstract

HomiCSx is an open-source FEniCSx framework that connects particulate geometry, periodic-conforming Gmsh meshing, material assignment, linear and finite-strain periodic homogenization, hooks, and essential outputs in one reproducible pipeline. Its supported core includes two- and three-dimensional cells, multiphase linear elasticity, user-defined hyperelastic energies with a built-in Neo-Hookean model, and generalized-Maxwell viscoelasticity. Verification combines analytical tests, constitutive checks, mesh refinement, regression gates, and cross-solver comparisons using independently constructed Abaqus models and conventional macroscopic quantities. Canonical linear discrepancies remain below 0.55%, and tested heterogeneous viscoelastic histories remain within 1.36%. HomiCSx targets inspectable, customizable computational-homogenization studies with an explicitly bounded support and maintenance scope.

Keywords: computational homogenization, finite element method, FEniCSx, representative volume element, composite materials, viscoelasticity

1. Motivation and significance

First-order computational homogenization obtains macroscopic constitutive response from microscopic boundary-value problems under scale separation [4, 5]. Its finite-element realization requires a chain of choices that

*Corresponding author

Email address: tahouniamirreza@gmail.com (Amir Reza Tahouni)

Nr	Code metadata description	Metadata
C1	Current code version	v1.0.1
C2	Permanent link to code/repository used for this code version	GitHub v1.0.1 tag
C3	Legal code license	MIT
C4	Code versioning system used	Git
C5	Software code languages, tools and services used	Python, NumPy, UFL, DOLFINx, PETSc/petsc4py, Basix, Gmsh API, MPI
C6	Compilation requirements, operating environments and dependencies	Conda; Python 3.10; DOLFINx 0.9.0; dolfinx_mpc 0.9.0; Linux or WSL2
C7	Link to developer documentation/manual	https://homicsx.readthedocs.io/en/latest/
C8	Support email for questions	tahouniamirreza@gmail.com

Table 1: SoftwareX code metadata for HomiCSx.

are often implemented in separate research scripts: constructing a representative cell, preserving periodic geometry, producing compatible opposite-boundary meshes, assigning phases, enforcing periodic constraints, solving multiple macroscopic load cases, and reducing microscopic fields to macroscopic quantities. This fragmentation makes it difficult to adapt a workflow without also weakening its traceability.

HomiCSx provides these steps through a common set of Python data objects and drivers built on DOLFINx and UFL [1, 2], with geometry and meshing provided by Gmsh [3]. Its main contribution is not a new homogenization theory. It is an extensible end-to-end implementation in which the geometry, discretization, constitutive response, macroscopic loading, nonlinear solution, and result extraction remain explicit and replaceable. This is useful for studies that require systematic microstructure generation or research-specific operations during a nonlinear homogenization history.

The software supports linear and finite-strain problems in two and three dimensions. Research-specific behavior can be introduced through custom material implementations, callable load cases, and typed hooks invoked at documented points in the nonlinear driver. The package therefore occupies a middle ground between one-off finite-element scripts and closed workflows

that offer limited intervention in the solution sequence.

1.1. Positioning relative to related software

HomiCSx complements rather than replaces broader research-software ecosystems. MicroStructPy emphasizes statistical microstructure generation and mesh export [8], while HomiCSx couples a narrower particulate generator directly to periodic mechanical solves. DAMASK provides mature grid and mesh solvers for crystal-plasticity and multiphysics homogenization [9]; HomiCSx instead targets compact, script-level customization of particulate FEniCSx workflows. MOOSE is a general parallel finite-element and multiphysics framework with periodic-boundary infrastructure [10], whereas HomiCSx packages domain-specific geometry, matching periodic meshing, constitutive hooks, homogenization loads, and macro outputs behind Python objects. HomiCSx does not claim their breadth, parallel scalability, or adoption. Its narrower contribution is an inspectable serial path joining these stages for linear, hyperelastic, and generalized-Maxwell RVE studies. Closer code-level comparators include micromacsfenics, which targets FEniCS/FEniCSx FE² and periodic or minimally constrained multi-scale problems, and FEniCS_homogenization, a collection of legacy-FEniCS linear elasticity scripts [11, 12]. They demonstrate that FEniCS-based homogenization implementations exist; the comparison set was therefore selected to cover both these direct implementations and broader geometry, constitutive, and multiphysics ecosystems. HomiCSx’s claimed distinction is the documented integration of particulate generation, periodic-conforming meshing, finite-strain/history-dependent workflows, hooks, and conventional validation rather than a claim to originate FEniCS homogenization.

2. Software description

2.1. Architecture and workflow

The workflow has five cooperating layers. The geometry layer creates seeded, periodic particulate cells or accepts explicitly constructed inclusions. The mesh layer builds the corresponding OpenCASCADE model, assigns physical tags, and generates triangular, tetrahedral, or supported two-dimensional all-quadrilateral meshes. Opposite boundaries are meshed with translated Gmsh periodic constraints so their nodes match. The material layer maps phase identifiers to constitutive objects. The finite-element layer constructs linear or nonlinear periodic fluctuation problems and applies

multipoint constraints. Finally, homogenization drivers execute standard or user-defined macroscopic load cases and return macroscopic histories and optional fields.

The core geometry types include circular and elliptical inclusions in 2D and spherical and ellipsoidal inclusions in 3D, with monodisperse or polydisperse sampling and clearance control. Periodic images retain source identity so that boundary-split particles are represented consistently. Geometry generation is seedable for reproducibility. Explicitly prescribed ellipses and ellipsoids may be rotated; random nonspherical packing remains axis-aligned. The publication-supported discretization path uses triangles, tetrahedra, or the tested two-dimensional all-quadrilateral option for linear and history-independent hyperelastic workflows. Generalized-Maxwell state updates require simplex cells. General three-dimensional hexahedral meshing is outside the current support boundary.

2.2. Homogenization formulation

For linear elasticity, the microscopic displacement is decomposed into a macroscopic affine field and a periodic fluctuation. Elementary macroscopic strain probes recover the effective stiffness from volume-averaged stress. In finite strain, the deformation gradient is decomposed as

$$\mathbf{F}(\mathbf{X}) = \bar{\mathbf{F}} + \nabla \tilde{\mathbf{u}}(\mathbf{X}), \quad (1)$$

where the fluctuation is periodic on opposite cell boundaries. The nonlinear driver solves incremental equilibrium with adaptive stepping and records volume-averaged first Piola–Kirchhoff stress, energy, deformation Jacobian, and optional finite-difference tangent information for history-independent materials.

Hyperelastic material objects provide a UFL strain-energy density and matching numerical post-processing methods. Compressible Neo-Hookean elasticity is the built-in, externally compared implementation; custom laws remain the user’s validation responsibility. For generalized-Maxwell branch i , let $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ and let $\mathbf{C}_{v,i}$ be its viscous metric. HomiCSx implements

$$\Psi_i = \frac{\mu_i}{2} [\text{tr}(\mathbf{C}\mathbf{C}_{v,i}^{-1}) - d - \ln \det(\mathbf{C}\mathbf{C}_{v,i}^{-1})], \quad (2)$$

with nonequilibrium stress $\mathbf{P}_i = \mu_i \mathbf{F}(\mathbf{C}_{v,i}^{-1} - \mathbf{C}^{-1})$. Starting from $\mathbf{C}_{v,i}^0 = \mathbf{I}$, a converged increment updates

$$\mathbf{C}_{v,i}^{n+1} = \alpha_i \mathbf{C}_{v,i}^n + (1 - \alpha_i) \mathbf{C}^{n+1}, \quad \alpha_i = \exp(-\Delta t / \tau_i). \quad (3)$$

The metrics are DG0 cellwise coefficients, consistent with the current cellwise deformation-gradient evaluation. Previous converged metrics remain fixed while global equilibrium is solved; trial state is committed only after convergence. The total stress is the equilibrium hyperelastic stress plus all branch stresses, so nonequilibrium stress enters the weak residual and its Newton Jacobian is obtained by automatic differentiation. Objectivity follows from dependence on $\mathbf{C}$ and an objective internal metric. Positive branch moduli and relaxation times are required. In the Abaqus comparison, τ_i is unchanged and the shear Prony ratio is $g_i = \mu_i/(\mu_\infty + \sum_j \mu_j)$ with zero bulk Prony ratio.

2.3. Customization and outputs

Users can define macroscopic deformation histories as Python callables and can derive new nonlinear material classes through the public material interface. Hooks provide controlled access before and after documented stages of the homogenization loop. They have explicit ordering and state scopes and can be used to collect fields, compute application-specific metrics, or implement additional workflow logic. Core outputs include effective stiffness, macro stress/strain/energy histories and Jacobian histories. Finite-difference tangents and reconstructed stress/energy XDMF fields are limited to history-independent materials; state-aware viscoelastic fields remain available to hooks. Figure 1 summarizes this division between the supported pipeline and its documented extension points.

3. Illustrative examples

The documentation demonstrates geometry generation, 2D and 3D linear solves, hyperelasticity, generalized-Maxwell response, and typed hooks. A single repository-level examples directory provides compact deterministic geometry, 2D/3D linear, hyperelastic, and heterogeneous viscoelastic workflows; all five serial scripts are executed by the test suite. The linear script creates a fixed geometry, generates a periodic-conforming mesh, assigns phase materials, and passes these objects to the linear driver. The viscoelastic script exercises nonlinear equilibrium, state evolution, and a typed macro-stress hook. The same separation of concerns allows a geometry or material implementation to be changed without replacing the full pipeline. Figure 2 illustrates the matching periodic boundary patterns produced by the meshing stage, while Figure 3 shows representative field output opened in ParaView.

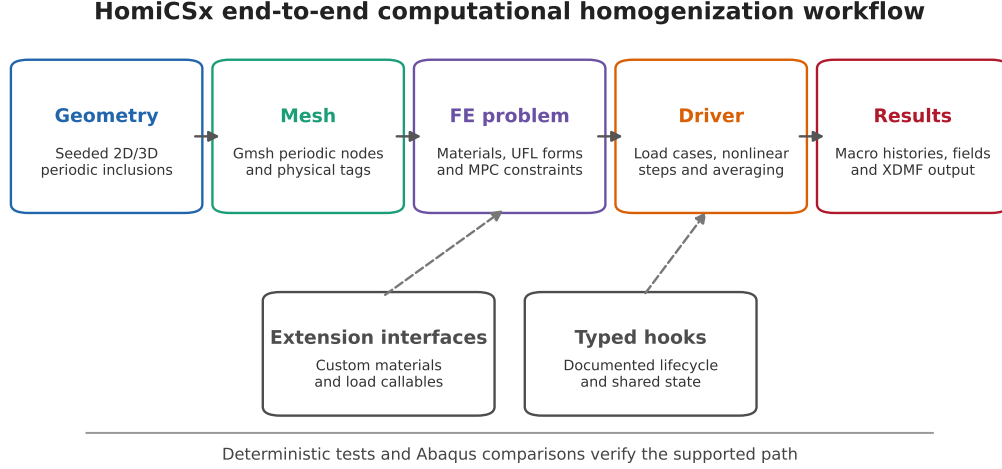


Figure 1: HomiCSx architecture and data flow. The supported workflow proceeds from geometry through periodic meshing, finite-element construction, homogenization, and result extraction. Custom materials, load callables, and typed hooks expose documented extension points.

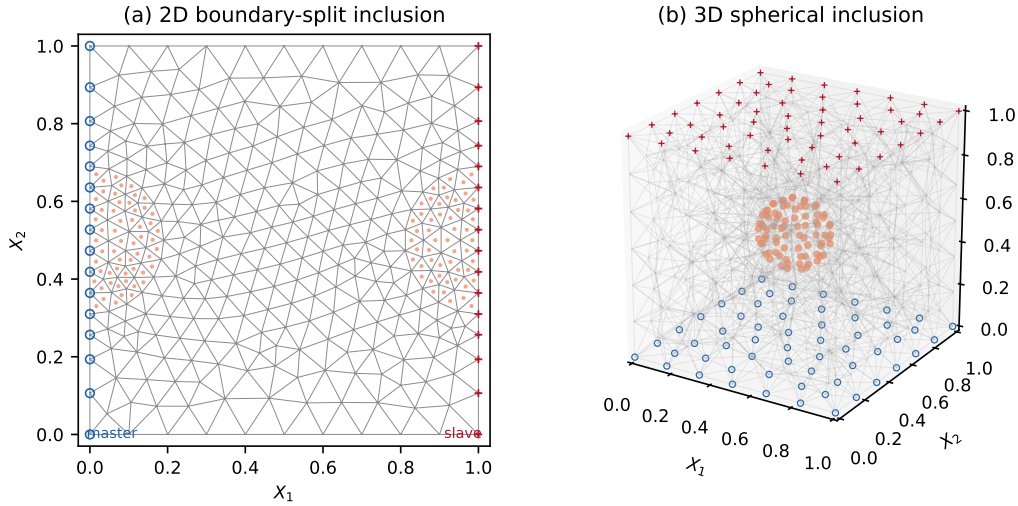


Figure 2: Representative periodic-conforming HomiCSx meshes. The 2D cell contains one physical circle split across the left/right boundary; the 3D cell contains a centered sphere. Blue circles mark master-face nodes and red crosses mark matching slave-face patterns.

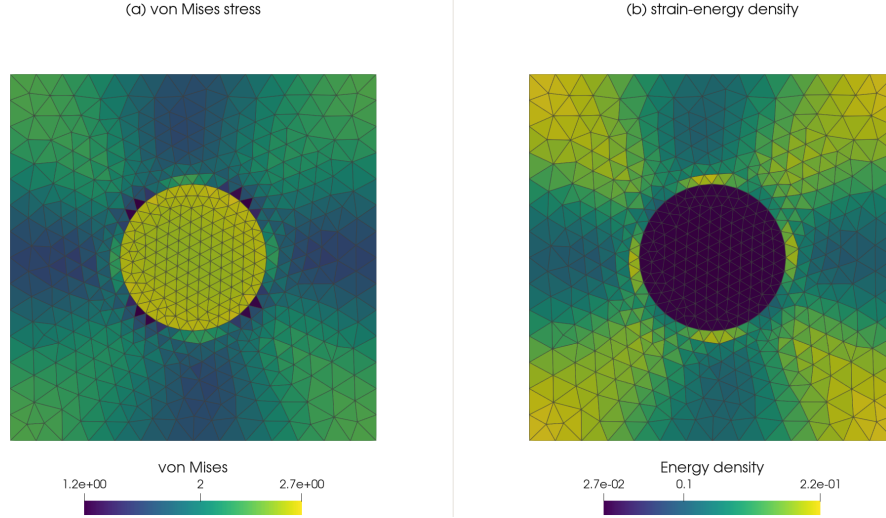


Figure 3: ParaView rendering of the final state at engineering shear $\gamma_{12} = 0.25$ for a two-phase Neo-Hookean cell. (a) von Mises stress; (b) strain-energy density. The field data are exported by HomiCSx and mesh edges are retained to distinguish the numerical field from a schematic.

3.1. Representative execution cost

For scale rather than as a performance benchmark, examples were run on one rank under WSL2 with Python 3.10.21 and DOLFINx 0.9.0 on an AMD Ryzen 9 9900X. OMP, OpenBLAS, and MKL thread counts were fixed to one. After one untimed warm-up, three independent process launches were measured with GNU `/usr/bin/time -f "%e %M" conda run -n homicsx_dev python <script>`. The 2D linear case (264 triangles; 298 unconstrained displacement DOFs; three load cases) took 1.80–1.83 s and 241 MiB peak RSS. The 3D linear case (1441 tetrahedra; 1206 DOFs; six load cases) took 2.16 s and 252–259 MiB. The 2D hyperelastic case (264 triangles; 298 DOFs; two increments) took 1.81–1.82 s and 241 MiB, and the three-increment viscoelastic case on the same mesh took 1.89–2.03 s and 243 MiB. Times include Conda process startup. These compact examples are workflow checks, not mesh-converged production benchmarks, and no parallel speedup is claimed.

4. Verification and validation

The automated test suite covers deterministic geometry behavior, tagging, periodic mesh construction, public API behavior, hook semantics, con-

stitutive parameter validation, and solver integration. Homogeneous 2D plane-strain and six-load-case 3D linear problems are compared with analytical isotropic stiffness. A deterministic 2D refinement gate reduces relative stiffness error from 2.543% on the coarse mesh to 0.211% on the refined mesh. Neo-Hookean first Piola stress is independently checked against numerical energy derivatives in 2D and 3D, and an end-to-end nonlinear patch recovers analytical macroscopic energy, stress, and mean Jacobian.

Because periodic-boundary enforcement is a consequential implementation choice in multiscale homogenization [6], the external validation suite includes a boundary-split geometry. It uses independently generated Abaqus models and only conventional macroscopic quantities. Three linear plane-strain cases—a homogeneous non-unit cell, a centered 20% circular inclusion, and a periodic boundary-split inclusion—have maximum stiffness, probe-stress, and probe-energy differences below 0.55%. Homogeneous finite simple shear agrees to within 0.00016% for macro energy and shear stress. The automated canonical gate regenerates both linear and homogeneous nonlinear HomiCSx results from the current checkout before comparison with the archived Abaqus reference.

Generalized-Maxwell validation compares complete macro-shear-stress relaxation histories. The homogeneous 2D and 3D curves agree with Abaqus to within 0.00063% and 0.00068% peak-normalized maximum error, respectively. For a viscoelastic matrix containing a hyperelastic inclusion, the centered and periodic-split 2D cases give 1.22% and 1.35% maximum curve errors. Automated fast/slow-rate regressions in 2D and 3D additionally require different converged heterogeneous fluctuation fields. The viscoelastic external curves are paired archived datasets; current-code regression is supplied by the analytical and heterogeneous rate tests rather than by rerunning Abaqus in CI. The external evidence does not establish pointwise field identity, all loading paths, or external validation of a heterogeneous 3D viscoelastic cell. Figure 4 shows the complete relaxation histories used for the viscoelastic comparisons rather than only their scalar error summaries.

5. Impact and reuse

HomiCSx enables a researcher to move from a seeded particulate description to homogenized linear, hyperelastic, or viscoelastic response in one inspectable workflow. Its hook mechanism is particularly relevant when a study needs quantities or interventions that are not general enough to be

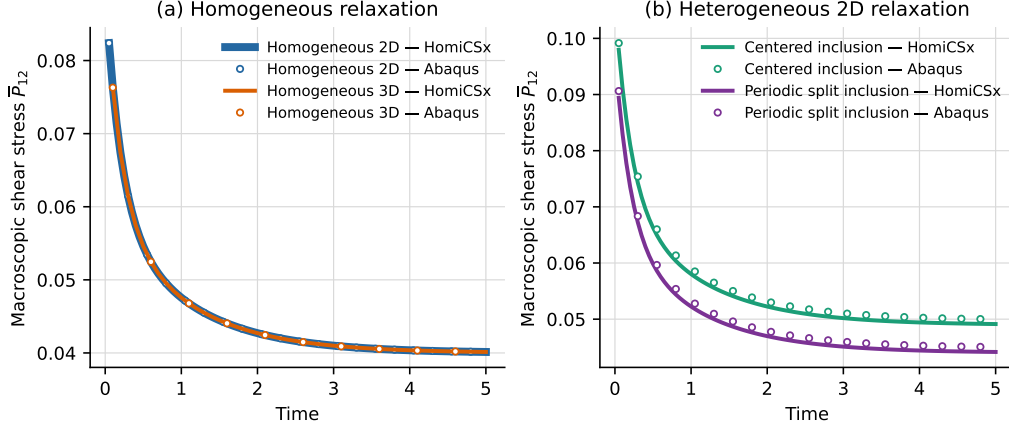


Figure 4: HomiCSx and Abaqus generalized-Maxwell shear-relaxation histories. Solid lines denote HomiCSx and open markers denote Abaqus. (a) Homogeneous 2D and 3D cells; (b) centered and periodic-split hyperelastic inclusions in a viscoelastic matrix.

long in a fixed post-processing interface. The software is already used by its author in a study of matrix localization in random periodic composites; that application uses custom material and field-processing logic while retaining the common geometry, mesh, equilibrium, and homogenization pipeline.

Potential reuse includes parametric studies of particulate composites, development and comparison of constitutive laws, generation of training data for reduced-order or surrogate models, and teaching reproducible RVE analysis. No claim of external adoption is made at this stage. The MIT license, public interfaces, tests, and archived releases are intended to allow independent use and continuation.

6. Limitations and maintenance

The tested environment is Linux, including Linux under WSL2, with Python 3.10, DOLFINx 0.9.0, and dolfinx_mpc 0.9.0. Two-dimensional FEM uses plane strain; plane stress is not implemented. The supported meshing path uses triangles, tetrahedra, and tested 2D all-quadrilateral meshes, although viscoelastic state updates require simplex cells; general 3D hexahedral meshing is not implemented. Imported meshes, independently oriented random nonspherical packing, overlapping-void and open-cell workflows, advanced visualization, and stochastic convenience modules are outside the

publication-supported core. The latter two modules are explicitly experimental.

HomiCSx is maintained as archival research software. No feature schedule or guaranteed support response is promised. Reproducibility, documentation, correctness fixes as resources permit, versioned releases, and preservation are prioritized. The permissive license allows community forks and continued development if active maintenance changes.

HomiCSx 1.x supports solver execution on one MPI rank. The drivers reject distributed communicators until cross-rank consistency is established and continuously tested.

7. Conclusions

HomiCSx provides a tested, inspectable path from periodic particulate geometry to homogenized linear, hyperelastic, and generalized-Maxwell response. Its principal value is integration with explicit research extension points rather than a new homogenization theory. The release combines a bounded supported core, conventional analytical and Abaqus evidence, reproducible environments, and candid limitations. This makes the software suitable for customized RVE studies while keeping its archival maintenance commitment realistic.

8. Availability and reproducibility

Source code: <https://github.com/AmirTahouni/HomiCSx>

Documentation: <https://homicsx.readthedocs.io/en/latest/>

License: MIT

Version described: 1.0.1

Version archive: <https://github.com/AmirTahouni/HomiCSx/tree/v1.0.1>

Concept DOI: <https://doi.org/10.5281/zenodo.22811693>

The repository contains version-constrained Conda environment specifications, installation instructions, automated tests, benchmark manifests, independently constructed Abaqus model runners scripts, compact reference results, and executable acceptance gates. Abaqus is not required to recompute comparisons from the committed compact results.

CRedit authorship contribution statement

Amir Reza Tahouni: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing—original draft, Writing—review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The software, validation manifests, scripts, and compact numerical results are available in the public repository and are preserved in the archived release identified above. Large proprietary Abaqus working databases are not required to evaluate the committed comparison results and are not distributed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used an OpenAI coding assistant to support code review, test and validation development, documentation editing, and manuscript drafting. After using this service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

- [1] I. A. Baratta, J. P. Dean, J. S. Dokken, et al., DOLFINx: The next generation FEniCS problem solving environment, preprint (2025). <https://doi.org/10.5281/zenodo.18101307>.

- [2] M. S. Alnæs, A. Logg, K. B. Ølgaard, M. E. Rognes, G. N. Wells, Unified Form Language: A domain-specific language for weak formulations of partial differential equations, *ACM Trans. Math. Softw.* 40 (2) (2014) Article 9, 1–37. <https://doi.org/10.1145/2566630>.
- [3] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities, *Int. J. Numer. Methods Eng.* 79 (11) (2009) 1309–1331. <https://doi.org/10.1002/nme.2579>.
- [4] M. G. D. Geers, V. G. Kouznetsova, W. A. M. Brekelmans, Multi-scale first-order and second-order computational homogenization of microstructures towards continua, *Int. J. Multiscale Comput. Eng.* 1 (4) (2003) 371–386. <https://doi.org/10.1615/IntJMultCompEng.v1.i4.40>.
- [5] S. Saeb, P. Steinmann, A. Javili, Aspects of computational homogenization at finite deformations: A unifying review from Reuss’ to Voigt’s bound, *Appl. Mech. Rev.* 68 (5) (2016) 050801. <https://doi.org/10.1115/1.4034024>.
- [6] P. Henyš, L. Čapek, J. Březina, Comparison of current methods for implementing periodic boundary conditions in multi-scale homogenisation, *Eur. J. Mech. A Solids* 78 (2019) 103825. <https://doi.org/10.1016/j.euromechsol.2019.103825>.
- [7] A. R. Tahouni, HomiCSx, version 1.0.1, GitHub (2026). <https://github.com/AmirTahouni/HomiCSx/tree/v1.0.1>.
- [8] K. A. Hart, J. J. Rimoli, MicroStructPy: A statistical microstructure mesh generator in Python, *SoftwareX* 12 (2020) 100595. <https://doi.org/10.1016/j.softx.2020.100595>.
- [9] M. Diehl, D. Wang, C. Liu, et al., Solving material mechanics and multi-physics problems of metals with complex microstructures using DAMASK, *Adv. Eng. Mater.* 22 (2020) 1901044. <https://doi.org/10.1002/adem.201901044>.
- [10] C. J. Permann, D. R. Gaston, D. Andrš, et al., MOOSE: Enabling massively parallel multiphysics simulation, *SoftwareX* 11 (2020) 100430. <https://doi.org/10.1016/j.softx.2020.100430>.
- [11] F. Rocha, micmacsfenics: a FEniCS-based implementation of two-level finite element simulations using computational homogenization, software repository (accessed 18 September 2026). <https://github.com/felipefr/micmacsfenics>.

- [12] B. Shrimali, FEniCS_homogenization: a collection of homogenization scripts for linear elasticity, software repository (accessed 18 September 2026). https://github.com/bhaveshrimali/FEniCS_homogenization.